



33

Observabilité API (Prometheus)

US3.6 · C6/C8

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Instrumenter l'API et exposer /metrics
- › Distinguer counter / gauge / histogram
- › Maîtriser la cardinalité (pas de donnée sensible en label)
- › Définir 5 SLI/SLO et écrire du PromQL

Déroulé de la séance

- 1 Théorie types, scrape, SLO, cardinalité
- 2 Travaux dirigés câblage, PromQL, charge
- 3 Autonomie tutorée gauge + extension B ou C
- 4 Bilan preuve + transition

PREUVE FINALE /metrics · gauge readiness · 5 SLI/SLO · p95 · charge canonique

1

Module 33 — les concepts clés

Théorie

■ Scénario atelier & enjeux

L'atelier se plaint que « l'API est lente » à 14 h. Sans métriques, aucune preuve ni cause. Avec /metrics, tu vois le p95 de latence et le trafic en direct.

Angle éthique / risque (C2) On mesure la technique, pas les personnes : aucune donnée personnelle dans les métriques. Observabilité n'est pas surveillance des opérateurs.

■ Contexte & outils — pourquoi ce module

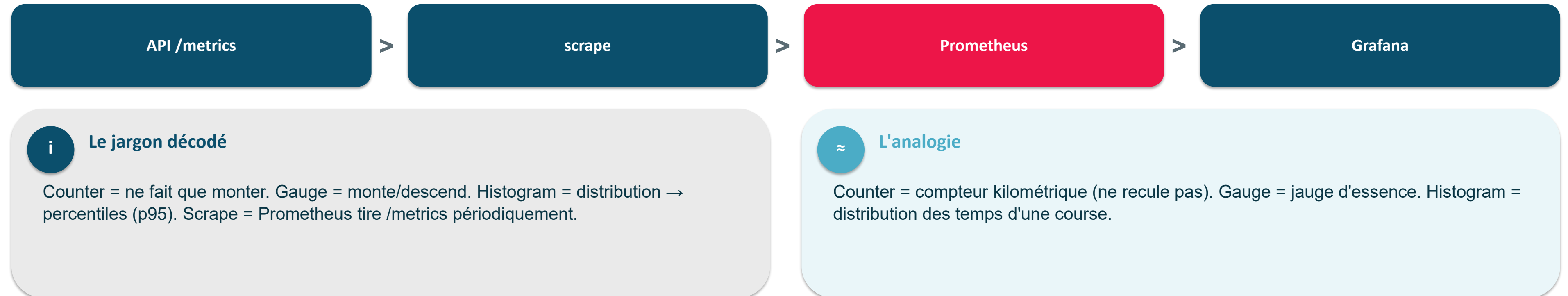
Contexte : un service en prod doit se regarder. Pourquoi : on MESURE latence, erreurs et trafic en continu.

- Prometheus : collecte les métriques (modèle PULL sur /metrics)
- counter / gauge / histogram : les 3 types de métriques
- histogram_quantile : calcule le p95 de latence
- SLI / SLO : objectifs de service (les 4 signaux d'or)
- /ready vs /health : prêt à servir vs simplement vivant

Objectif du module Des métriques exploitables, avec une cardinalité maîtrisée.

■ Mesurer le service : counter, gauge, histogram

Trois types de métriques suffisent à tout décrire : ce qui monte, ce qui fluctue, ce qui se distribue.



EN UNE PHRASE L'API expose /metrics ; Prometheus vient le scraper, Grafana l'affiche.

■ Le piège n°1 : la cardinalité

Chaque valeur de label crée une série : un label à forte cardinalité fait exploser Prometheus.

label route — sain

- ~5 valeurs
- quelques séries ✓

label machine_id

- 15 ici · plus en production
- explosion de séries X

! Attention — piège

Un label « innocent » comme machine_id peut générer des milliers de séries et faire tomber le monitoring. La cardinalité se réfléchit AVANT.

! À retenir

5 SLO v0 : p95 · erreurs 5xx · readiness modèle (up × gauge) · débit · 401.

EN UNE PHRASE Labels à faible cardinalité uniquement — jamais d'id ni de clé (fuite).

■ L'instrumentation, en vrai (déjà câblée)

L'API expose /metrics ; Prometheus le scrape. Counter/Gauge/Histogram, cardinalité maîtrisée.

```
# api/main.py · métriques métier et readiness
PRED = Counter("indusense_predictions_total", "", ["decision"])
MODEL = Gauge("indusense_model_loaded", "modèle chargé: 1/0")
# lifespan : MODEL.set(1) après load_bundle ; sinon MODEL.set(0)
Instrumentator(should_group_status_codes=False).instrument(app).expose(app)
# prometheus.yml : job_name: indusense-api · metrics_path: /metrics
# cible du réseau compose : api:8000
```

! Attention — piège

Jamais machine_id / clé / timestamp en label : explosion de séries + fuite de données.

! À retenir

5 SLO v0 : p95, erreurs 5xx, readiness up × gauge, débit predict, taux de 401.

EN UNE PHRASE Instrumenter le service + SLO chiffrés = la preuve C6/C8.

■ À retenir en 3 points

- /metrics expose latence, trafic et erreurs.
- SLI = ce qu'on mesure ; SLO = l'objectif visé.
- On suit le p95, pas la moyenne (qui ment).

2

Module 33 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — instrumenter & métrique custom

```
Instrumentator(should_group_status_codes=False).instrument(app).expose(app) # /metrics  
PRED = Counter("indusense_predictions_total", "", ["decision"]) # faible cardinalité
```

✓ Point de contrôle `curl.exe http://localhost:8000/metrics` répond ; label = decision (pas machine_id).

TP — SLO & PromQL

```
# latence p95 · job canonique
histogram_quantile(0.95, sum by (le) (
  rate(http_request_duration_seconds_bucket{job="indusense-api"}[5m])))
```

Charge canonique : `uv run --with locust==2.44.4 locust -f $m33Locustfile --headless -u 20 -r 5 -t 30s --host http://localhost:8000`

3

Module 33 — vous avancez, le formateur reste disponible

Autonomie tutorée

Autonomie tutorée — enrichir l'observabilité

Choisissez *UN* axe ; le formateur reste disponible.

-  Preuve obligatoire · Gauge modèle chargé (0/1) cohérente avec /ready
-  Extension au choix · B PromQL 5xx OU C labo vision / PatchCore

 Preuve & garde-fous

Montrer la gauge. B ou C la complète ; aucun seuil métier/coût déduit ; aucun identifiant en label.

■ Definition of Done & transition

● DoD : /metrics · gauge readiness · 5 SLI/SLO · PromQL p95 · Locust canonique

Transition → Module 34 Visualiser, alerter, jouer les runbooks avant le Game Day.